CS 7643: Deep Learning Solutions
Fall 2026
Problem Set 3

Author: Venkata Aashutosh Aripirala
GTID: 904173725

October 18, 2026

Instructions

1. We will be using Gradescope to collect your assignments. Please read the following instructions for submitting to Gradescope carefully!

   - For the **HW3 Theory** component on Gradescope, upload one single PDF containing the answers to all the theory questions. **However, the solution to each problem/subproblem must be on a separate page. When submitting to Gradescope, please make sure to mark the page(s) corresponding to each problem/subproblem.** Questions with misassigned pages will be penalized by up to 1 point.
   - For the **HW3 Code** on Gradescope, please use the `collect_submission.py` script provided and upload **hw3_code_submission.zip**
   - For the **HW3 Notebook** component on Gradescope, please use the `collect_submission.py` script provided and upload the resulting **hw3_notebook_submission.pdf** on Gradescope.
   - Note: This is a large class and Gradescope's assignment segmentation features are essential. Failure to follow these instructions may result in parts of your assignment being penalized. We will not entertain regrading requests for failure to follow instructions.

2. LaTeX'd solutions are strongly encouraged (solution template available on Canvas—HW3 Theory), but scanned handwritten copies are acceptable. Hard copies are **not** accepted.

3. We generally encourage you to collaborate with other students.

   You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and *not* as a group activity. Please list the students you collaborated with.

1 Collaborators [0.5 points]

Please list your collaborators and assign this list to the corresponding section of the outline on Gradescope. If you don't have any collaborators, please write 'None' and assign it to the corresponding section of the Gradescope submission regardless. Not assigning this question on Gradescope or leaving it blank will be marked as zero.

Solution

None

2 Logic and XOR

1. **[1 point]** Implement AND and OR for pairs of binary inputs using a single linear threshold neuron with weights $\mathbf{w} \in \mathbb{R}^2$, bias $b \in \mathbb{R}$, and $\mathbf{x} \in \{0,1\}^2$:

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T\mathbf{x} + b < 0 \end{cases} \tag{1}$$

That is, find $\mathbf{w}_{\texttt{AND}}$ and $b_{\texttt{AND}}$ such that

x_1	x_2	$f_{\texttt{AND}}(\mathbf{x})$
0	0	0
0	1	0
1	0	0
1	1	1

Also find $\mathbf{w}_{\texttt{OR}}$ and $b_{\texttt{OR}}$ such that

x_1	x_2	$f_{\texttt{OR}}(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	1

You do not need to show your work for this question. You can just provide the specific values for $\mathbf{w}$ and b.

Solution

For the **AND** gate:

$$\mathbf{w}_{\texttt{AND}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b_{\texttt{AND}} = -1.5$$

For the **OR** gate:

$$\mathbf{w}_{\texttt{OR}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b_{\texttt{OR}} = -0.5$$

2. **[1 point]** Consider the XOR function

x_1	x_2	$f_{\texttt{XOR}}(x)$
0	0	0
0	1	1
1	0	1
1	1	0

Prove that XOR can NOT be represented using a linear model with the same form as (1). Graphical/visual proofs will not receive full credit.

[*Hint:* To see why, plot the examples from above in a plane and think about drawing a linear boundary that separates them.]

Solution

We prove this by contradiction.

Assume that there exists a weight vector $\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^2$ and a bias $b \in \mathbb{R}$ such that the model

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0 & \text{if } w_1 x_1 + w_2 x_2 + b < 0 \end{cases}$$

correctly represents $f_{\texttt{XOR}}(\mathbf{x})$ for all inputs $\mathbf{x} \in \{0,1\}^2$.

Based on the XOR truth table, the following four conditions must hold simultaneously:

$$\mathbf{x} = (0,0) \implies f(\mathbf{x}) = 0 \implies w_1(0) + w_2(0) + b < 0 \implies b < 0 \tag{2}$$
$$\mathbf{x} = (0,1) \implies f(\mathbf{x}) = 1 \implies w_1(0) + w_2(1) + b \geq 0 \implies w_2 + b \geq 0 \tag{3}$$
$$\mathbf{x} = (1,0) \implies f(\mathbf{x}) = 1 \implies w_1(1) + w_2(0) + b \geq 0 \implies w_1 + b \geq 0 \tag{4}$$
$$\mathbf{x} = (1,1) \implies f(\mathbf{x}) = 0 \implies w_1(1) + w_2(1) + b < 0 \implies w_1 + w_2 + b < 0 \tag{5}$$

Summing inequalities (3) and (4) gives:

$$(w_2 + b) + (w_1 + b) \geq 0 \implies w_1 + w_2 + 2b \geq 0 \tag{6}$$

Rearranging (6):

$$w_1 + w_2 + b \geq -b \tag{7}$$

From inequality (2), we have $b < 0$, which implies $-b > 0$. Substituting this into (7):

$$w_1 + w_2 + b > 0 \tag{8}$$

However, this directly contradicts inequality (5), which requires $w_1 + w_2 + b < 0$.

Since no such real values (w_1, w_2, b) can satisfy all four constraints simultaneously, the XOR function cannot be represented by a single linear threshold neuron. □

3 Recurrent Neural Networks and Transformers

3. **[LSTM for parity function: 4 points]** Let us recall different gates in an LSTM Network. First gate is the "forget gate layer":

$$f_t = \sigma(W_f \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_f) \tag{9}$$

where f_t is the output of forget gate, W_f is the weight matrix, h_{t-1} is the hidden state of step *t-1*, x_t is the current input and b_f is the bias value.
Next we have "input gate layer":

$$i_t = \sigma(W_i \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_i) \tag{10}$$

$$\tilde{C}_t = \tanh(W_C \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_C) \tag{11}$$

where i_t decides which values we will update and $\tilde{C}_t$ are the new candidate values that could be added to the cell state. Next we have new cell state candidate values:

$$C_t = C_{t-1} \times f_t + \tilde{C}_t \times i_t \tag{12}$$

Finally, we have the output and hidden state

$$o_t = \sigma(W_o \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_o) \tag{13}$$

$$h_t = o_t \times \tanh(C_t) \tag{14}$$

Design an LSTM Network for the bit parity problem, which returns 0 if there is an even number of 1s in an input bit string, and 1 otherwise. Specifically,

(a) provide values for W_f, b_f, W_i, b_i, W_C, b_C, W_o and b_o such that the cell state C_t stores the parity of an input bit string, assuming x, h, f, i, o, b, and C are all scalars.

(b) briefly summarize how your LSTM implements the logic to determine bit parity

(c) explain how you obtained your values

Please mention any assumptions you make. For this problem, you can assume below for Sigmoid and tanh function:

$$\sigma(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases} \tag{15}$$

$$\tanh(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & x = 0 \\ -1, & \text{if } x < 0 \end{cases} \tag{16}$$

Hint: Recall that XOR of x and y can be represented as $(x \wedge \bar{y}) \vee (\bar{x} \wedge y)$ which you implemented above. Think about how you can leverage this information for equation (12).

Solution

Assumptions

- Initial states: $C_0 = 0$ and $h_0 = 0$. (Even parity corresponds to value 0, odd parity corresponds to 1).
- Binary inputs: $x_t \in \{0, 1\}$.
- Weight matrices are row vectors of dimension 1×2:

$$W = \begin{bmatrix} W_{*,h} & W_{*,x} \end{bmatrix} \implies W \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} = W_{*,h} h_{t-1} + W_{*,x} x_t$$

(a) Parameter Values

- **Forget Gate:**
$$W_f = \begin{bmatrix} 0 & -2 \end{bmatrix}, \quad b_f = 1$$
- **Input Gate:**
$$W_i = \begin{bmatrix} -2 & 2 \end{bmatrix}, \quad b_i = -1$$
- **Candidate Cell State:**
$$W_C = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad b_C = 1$$
- **Output Gate:**
$$W_o = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad b_o = 1$$

(b) Logic Summary

The parity problem is an incremental XOR operation between the previous parity state $h_{t-1} \in \{0, 1\}$ and the current input bit $x_t \in \{0, 1\}$:

$$C_t = h_{t-1} \oplus x_t = (h_{t-1} \wedge \bar{x}_t) \vee (\bar{h}_{t-1} \wedge x_t)$$

In an LSTM, the update equation is:

$$C_t = C_{t-1} \cdot f_t + \tilde{C}_t \cdot i_t$$

Because the output gate always opens ($o_t = 1$), we have $\tanh(C_t) = C_t \implies h_t = C_t \in \{0, 1\}$. Thus, $C_{t-1} = h_{t-1}$.

- When $h_{t-1} = 1$: $C_{t-1} \cdot f_t = 1 \cdot f_t = f_t$. To satisfy the XOR logic, C_t must keep the value 1 if and only if $x_t = 0$. Hence, we choose the forget gate so that $f_t = 1$ when $x_t = 0$ and $f_t = 0$ when $x_t = 1$. Since $C_{t-1} = h_{t-1}$, the product $C_{t-1} f_t$ implements the term $(h_{t-1} \wedge \bar{x}_t)$.
- When $h_{t-1} = 0$: $C_{t-1} \cdot f_t = 0$, so $C_t = \tilde{C}_t \cdot i_t$. Setting $\tilde{C}_t = 1$ and letting i_t act as the term $(\bar{h}_{t-1} \wedge x_t)$, we have $i_t = 1$ when $(h_{t-1}, x_t) = (0, 1)$, and 0 otherwise.

Because the two conditions $(h_{t-1} = 1 \wedge x_t = 0)$ and $(h_{t-1} = 0 \wedge x_t = 1)$ are mutually exclusive, $C_t = f_t + i_t = h_{t-1} \oplus x_t \in \{0, 1\}$.

(c) Derivation of Parameters

(a) **Output Gate ($o_t = 1$):** We want $o_t = \sigma(0 \cdot h_{t-1} + 0 \cdot x_t + b_o) = 1$ for all inputs. Using $b_o = 1 > 0$ yields $o_t = 1$. Since $C_t \in \{0, 1\}$, $\tanh(0) = 0$ and $\tanh(1) = 1$, giving $h_t = o_t \tanh(C_t) = C_t$.

(b) **Candidate State ($\tilde{C}_t = 1$):** Choosing $W_C = [0, 0]$ and $b_C = 1 > 0$ gives $\tilde{C}_t = \tanh(1) = 1$ at all steps.

(c) **Forget Gate:** Since $C_{t-1} = 0$ zeroes out $C_{t-1} f_t$ automatically, when $h_{t-1} = 1$, we require $f_t = 1$ if $x_t = 0$ and $f_t = 0$ if $x_t = 1$:

$$\begin{cases} (h_{t-1}, x_t) = (1, 0) \implies W_{f,h}(1) + W_{f,x}(0) + b_f > 0 \\ (h_{t-1}, x_t) = (1, 1) \implies W_{f,h}(1) + W_{f,x}(1) + b_f \leq 0 \end{cases}$$

Setting $W_f = [0, -2]$ and $b_f = 1$:

- If $x_t = 0$: $0(h_{t-1}) - 2(0) + 1 = 1 > 0 \implies f_t = 1$.
- If $x_t = 1$: $0(h_{t-1}) - 2(1) + 1 = -1 \leq 0 \implies f_t = 0$

s This choice $W_f = \begin{bmatrix} 0 & -2 \end{bmatrix}, b_f = 1$ yields $f_t = 1$ whenever $x_t = 0$, meaning $C_t = C_{t-1} \cdot 1 = C_{t-1}$ when $x_t = 0$.

(d) **Input Gate ($i_t = \bar{h}_{t-1} \wedge x_t$):** We require $i_t = 1$ strictly when $(h_{t-1}, x_t) = (0, 1)$, and 0 otherwise:

$$\begin{cases} (0, 0) : b_i \leq 0 \\ (0, 1) : W_{i,x} + b_i > 0 \\ (1, 0) : W_{i,h} + b_i \leq 0 \\ (1, 1) : W_{i,h} + W_{i,x} + b_i \leq 0 \end{cases}$$

Setting $W_i = \begin{bmatrix} -2 & 2 \end{bmatrix}$ and $b_i = -1$:

- $(0, 0) : -1 \leq 0 \implies i_t = 0$
- $(0, 1) : 2(1) - 1 = 1 > 0 \implies i_t = 1$
- $(1, 0) : -2(1) - 1 = -3 \leq 0 \implies i_t = 0$
- $(1, 1) : -2(1) + 2(1) - 1 = -1 \leq 0 \implies i_t = 0$

Together with $W_f = \begin{bmatrix} 0 & -2 \end{bmatrix}, b_f = 1$, the transition table verifies as:

h_{t-1}	x_t	f_t	i_t	$\tilde{C}_t$	$C_t = h_{t-1} f_t + \tilde{C}_t i_t$	h_t
0	0	1	0	1	$0(1) + 1(0) = 0$	0
0	1	0	1	1	$0(0) + 1(1) = 1$	1
1	0	1	0	1	$1(1) + 1(0) = 1$	1
1	1	0	0	1	$1(0) + 1(0) = 0$	0

4. **[When to Stop in Beam Search: 2 points]** Beam Search is a widely-used technique for decoding the most likely sequence from sequence models. But it is difficult to decide when to stop beam search to obtain optimality because hypotheses can finish in different steps. In this question, we will develop a formal understanding of the stopping criteria in beam search.

   Suppose we are generating a sequence $\mathbf{y} = (y_1, y_2, \ldots, y_T)$ conditioned on input $\mathbf{x}$ using beam search with beam size b, which is the number of sequences we are exploring in parallel. We will sample next tokens in the sequence using the distribution defined by $p(y_t|\mathbf{x}, y_{<i})$, which can be thought of as an autoregressive sequence model.

   - A sequence $\mathbf{y}$ is **complete** if it ends with `</s>`:

$$\text{COMPLETED}(\mathbf{y}) = \text{true} \iff y_{|\mathbf{y}|} = \texttt{</s>}$$

- The **score** $s(\mathbf{y})$ of a sequence $\mathbf{y}$ is the sum of the token log probabilities:

$$s(\mathbf{y}) = \sum_{i=1}^{|\mathbf{y}|} \log p(y_i \mid \mathbf{x}, y_{<i})$$

- Let B_t be the set of the top-b active beams at step t, where each element is $\langle \mathbf{y}, s(\mathbf{y}) \rangle$.

$$B_t = \text{top}^b \Big\{ \langle \mathbf{y} \circ y_t,\ s(\mathbf{y}) + \log p(y_t \mid \mathbf{x}, \mathbf{y}) \rangle \ : \langle \mathbf{y}, s(\mathbf{y}) \rangle \in B_{t-1},\ y_t \in \mathcal{V} \Big\}$$

  where $\mathcal{V}$ is the vocabulary and top^b selects the b highest-scoring candidates. Assume we start sampling from an empty sequence such that $B_0 = \{\langle \{\}, 0 \rangle\}$.

- $s^{(t)}_{\text{best}}$ = score of the best *completed* sequence found up to step t:

$$s^{(t)}_{\text{best}} = \max \left\{ s(\mathbf{y}) \mid \langle \mathbf{y}, s(\mathbf{y}) \rangle \in \bigcup_{j \le t} B_j,\ \text{COMPLETED}(\mathbf{y}) = \text{true} \right\}$$

  Suppose at step t, $s^{(t)}_{\text{best}}$ is defined (at least one complete sequence exists) and that the current highest scoring beam in B_t (*i.e.* $\mathbf{y}^1$) scores worse than or equal to $s^{(t)}_{\text{best}}$ (*i.e.* $s^1 \le s^{(t)}_{\text{best}}$). **For this question, explain why no future completed sequence $\mathbf{y}^*$ (of length $>= t+1$) will have a score greater than $s^{(t)}_{\text{best}}$.**

Solution

We want to prove why no future completed sequence $\mathbf{y}^*$ of length $|\mathbf{y}^*| \ge t+1$ can attain a score strictly greater than $s^{(t)}_{\text{best}}$, given that the highest-scoring candidate in the active beam B_t satisfies:

$$s(\mathbf{y}^1) \le s^{(t)}_{\text{best}}$$

By definition, the score of any candidate sequence $\mathbf{y} = (y_1, y_2, \ldots, y_k)$ is the cumulative sum of log probabilities:

$$s(\mathbf{y}) = \sum_{i=1}^{k} \log p(y_i \mid \mathbf{x}, y_{<i})$$

Since $p(y_i \mid \mathbf{x}, y_{<i})$ is a valid conditional probability, $0 \leq p(y_i \mid \mathbf{x}, y_{<i}) \leq 1$. Consequently, every log probability term is non-positive:

$$\log p(y_i \mid \mathbf{x}, y_{<i}) \leq 0, \quad \forall i$$

For any extended sequence $\mathbf{y}' = \mathbf{y} \circ y_{k+1} \circ \cdots \circ y_m$ $(m > k)$, its score is:

$$s(\mathbf{y}') = s(\mathbf{y}) + \sum_{j=k+1}^{m} \log p(y_j \mid \mathbf{x}, y_{<j}) \leq s(\mathbf{y})$$

Thus, extending any sequence can only decrease (or at best preserve) its cumulative log-likelihood score.

Any candidate sequence $\mathbf{y}^*$ completed at some future step $T \geq t+1$ must originate by extending one of the active beams present in B_t:

$$\exists \mathbf{y} \in B_t \quad \text{such that} \quad \mathbf{y}^* = \mathbf{y} \circ y_{t+1} \circ \cdots \circ y_T$$

Applying the monotonic non-increasing property:

$$s(\mathbf{y}^*) \leq s(\mathbf{y})$$

By definition, $\mathbf{y}^1$ is the highest-scoring beam in B_t, which implies:

$$s(\mathbf{y}) \leq s(\mathbf{y}^1), \quad \forall \mathbf{y} \in B_t$$

Combining these inequalities with the given stopping condition $s(\mathbf{y}^1) \leq s_{\text{best}}^{(t)}$:

$$s(\mathbf{y}^*) \leq s(\mathbf{y}) \leq s(\mathbf{y}^1) \leq s_{\text{best}}^{(t)}$$

Therefore, $s(\mathbf{y}^*) \leq s_{\text{best}}^{(t)}$. No future completed sequence extending a beam in B_t can achieve a score greater than the best completed sequence already found. Thus, the stated stopping condition is valid for terminating the current beam search.

5. **[Exploding Gradients: 2 points]** Learning long-term dependencies in recurrent networks suffers from a particular numerical challenge – gradients propagated over many time-steps tend to either 'vanish' (converge to 0) or 'explode' (diverge to infinity). To study this problem in a simple setting, consider the following recurrence relation without any nonlinear activation function or input x:

$$h_t = Wh_{t-1} \tag{17}$$

where W is a weight sharing matrix for recurrent relation at any time t. Let $\lambda_1, ..., \lambda_n$ be the n distinct eigenvalues of the weight matrix $W \in \mathbb{C}^{n\times n}$. Its spectral radius $\rho(W)$ is defined as:

$$\rho(W) = \max\{|\lambda_1|, ..., |\lambda_n|\} \tag{18}$$

Assuming the initial hidden state is h_0, write h_t in terms of h_0 and W and explain the role of the eigenvalues of W in determining the 'vanishing' or 'exploding' property as $t \gg 0$ (t grows very large).

Solution

Given the linear recurrent relation:

$$h_t = Wh_{t-1}$$

We can write this recurrence recursively going from step t down to 0:

$$\begin{aligned} h_1 &= Wh_0 \\ h_2 &= Wh_1 = W(Wh_0) = W^2h_0 \\ &\vdots \\ h_t &= W^th_0 \end{aligned}$$

Assume $W \in \mathbb{C}^{n\times n}$ has n distinct eigenvalues $\lambda_1, \ldots, \lambda_n$ with corresponding linearly independent eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$. Then through eigen-value decomposition, W can be decomposed as:

$$W = V\Lambda V^{-1}$$

where $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$ and $V = [\mathbf{v}_1, \ldots, \mathbf{v}_n]$. Raising W to the t-th power yields:

$$W^t = V\Lambda^t V^{-1} = V \begin{bmatrix} \lambda_1^t & & 0 \\ & \ddots & \\ 0 & & \lambda_n^t \end{bmatrix} V^{-1}$$

Since the eigenvectors form a basis of $\mathbb{C}^n$, any initial hidden state h_0 can be represented uniquely as a linear combination of eigenvectors:

$$h_0 = \sum_{i=1}^{n} c_i\mathbf{v}_i, \quad c_i \in \mathbb{C}$$

Multiplying by W^t:

$$h_t = W^t h_0 = \sum_{i=1}^{n} c_i W^t \mathbf{v}_i = \sum_{i=1}^{n} c_i \lambda_i^t \mathbf{v}_i$$

The spectral radius of W is defined as $\rho(W) = \max_{1 \le i \le n} |\lambda_i|$. As $t \gg 0$ ($t \to \infty$), the asymptotic behavior of each term λ_i^t depends entirely on the magnitude of the eigenvalues:

- **Vanishing Gradients / Activations ($\rho(W) < 1$):** If $|\lambda_i| < 1$ for all $i \in \{1, \dots, n\}$, then for every component:

  $$\lim_{t \to \infty} |\lambda_i|^t = 0 \implies \lim_{t \to \infty} h_t = \mathbf{0}$$

  Similarly, when computing backpropagation through time, the gradient of a loss $\mathcal{L}_T$ at time T with respect to state h_t involves the Jacobian product:

  $$\frac{\partial h_T}{\partial h_t} = W^{T-t}$$

  When $\rho(W) < 1$, as the temporal gap $(T - t)$ becomes large, $W^{T-t} \to 0$. Gradients decay exponentially to zero, preventing the network from learning long-term temporal dependencies.

- **Exploding Gradients / Activations ($\rho(W) > 1$):** If there exists at least one eigenvalue such that $|\lambda_k| > 1$, then for any initial condition h_0 with non-zero projection along $\mathbf{v}_k$ ($c_k \neq 0$):

  $$|\lambda_k|^t \to \infty \quad \text{as } t \to \infty \implies \|h_t\| \to \infty$$

  In BPTT, $\|W^{T-t}\| \geq (\rho(W))^{T-t} \to \infty$, causing the backpropagated gradients to grow exponentially large, resulting in numerical overflow (`NaN` / `Inf`) and unstable parameter updates.

- **Stability around ($\rho(W) = 1$):** When $\rho(W) = 1$ and all eigenvalues satisfy $|\lambda_i| \leq 1$, the powers W^t remain bounded under the distinct-eigenvalues assumption. Importantly though, boundedness of W^t of alone does not guarantee that they are preserved at a constant magnitude or that long-term learning is stable.

4 Paper Review

For this homework's paper review section, we will read **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. The paper can be viewed here.
The evaluation rubric for this section is as follows:

6. **[2 points]** Briefly summarize the key contributions, strengths and weaknesses of this paper.

Solution

FlashAttention introduces an IO-aware exact attention algorithm designed to address the memory bandwidth bottleneck in standard Transformer attention.

Contributions The authors identify that standard attention incurs substantial data-transfer overhead between slow GPU high-bandwidth memory (HBM) and fast on-chip SRAM. They introduce a tiled attention computation using online softmax, avoiding materialization of the full $(N \times N)$ attention matrix in HBM. They also develop an IO-aware backward pass that recomputes intermediate attention quantities rather than storing the entire attention matrix, reducing memory consumption and data transfers.

Strengths

- **Exact & Memory Efficient:** Unlike approximate methods, FlashAttention computes mathematically exact attention upto floating point rounding differences while reducing HBM accesses from quadratic $O(N^2)$ to sub-quadratic $O(N^2d^2/M)$ "under the paper's assumptions", while retaining the $O(N^2d)$ arithmetic cost of dense attention, where (N) is the sequence length, (d) is the head dimension, and (M) is the available SRAM capacity. Its memory usage is linear in sequence length for fixed head dimensions, rather than quadratic as in standard attention implementations that materialize the attention matrix.
- **Downstream Impact:** The paper demonstrates substantial wall-clock speedups on several Transformer workloads, including a $(3\times)$ speedup on GPT-2 at sequence length 1K and a $(2.4\times)$ speedup on Long Range Arena tasks. FlashAttention also enables experiments with longer contexts, including sequences of 16K and 64K tokens in specific Path-X and Path-256 benchmarks.

Weaknesses

- **Hardware Specificity:** Implementing efficient SRAM tiling requires low-level, hardware-specific kernel optimization (e.g., custom CUDA/Triton kernels), making portability across non-NVIDIA custom accelerators non-trivial.
- **SRAM Size Dependence:** Performance depends on available on-chip SRAM, memory bandwidth, head dimension, and tile configuration. Consequently, the performance gains vary across hardware platforms and workloads, and the implementation must select suitable tile sizes to use on-chip resources efficiently.

7. **[2 points]** What is your personal takeaway from this paper? This could be expressed either in terms of relating the approaches adopted in this paper to your traditional understanding of explainability techniques, or potential future directions of research in the area which the authors haven't addressed, or anything else that struck you as being noteworthy.

Solution

My key takeaway is that algorithmic design in modern deep learning cannot treat computation in isolation from the hardware memory hierarchy. While algorithmic complexity is often analyzed in terms of FLOPs, FlashAttention demonstrates that execution time can also be dominated by memory IO bottlenecks. By trading additional computation, such as recomputing intermediate attention quantities during the backward pass, for reduced HBM read/write traffic, the authors achieve substantial practical speedups while retaining the mathematical exactness of attention.

Looking forward, this IO-aware paradigm opens several compelling directions:

- **Cross-Layer Operator Fusion:** Extending fine-grained memory tiling beyond self-attention into entire Transformer blocks (fusing LayerNorm, projections, MLP layers, and residual connections into unified SRAM tiles) to minimize intermediate DRAM writes.
- **Hardware-Co-Design for Non-Uniform Memory:** Adapting IO-aware primitives for heterogeneous architectures, distributed multi-node pipeline stages, and decentralized inference where memory latency is distributed across interconnects.

Overall, the work underscores that bridging system-level architectural realities with core deep learning algorithms often yields more practical breakthroughs than asymptotic algorithmic changes alone.